

Tecnológico de Monterrey

Evidencia.1.Actividad Integradora 1

Álvaro González Martínez - A01646343
Valeria Fernanda Hernández Avilán - A01640968

Análisis y diseño de algoritmos avanzados (Gpo 603)
Jueves, 1 de Octubre, 2026

Tecnológico de Monterrey

Algoritmos aplicados y complejidad computacional

1. KMP (Knuth-Morris-Pratt)
Para la primera parte del problema utilizamos el algoritmo KMP para buscar cada código malicioso (mcode) dentro de las transmisiones.

Primero se construimos un arreglo LPS (Longest Prefix Suffix) para el código que se desea buscar. Este arreglo almacena información sobre prefijos y sufijos que coinciden dentro del patrón. Cuando durante la búsqueda se encuentran dos caracteres diferentes, KMP utiliza el LPS para determinar qué parte de la coincidencia anterior puede reutilizarse, evitando regresar en la transmisión y repetir comparaciones innecesarias.

En el programa que desarrollamos, construirLPS() genera este arreglo y buscarKMP() lo utiliza para realizar la búsqueda.

Si n es el tamaño de la transmisión y m el tamaño del código malicioso:
- Construcción del LPS: O(m)
- Búsqueda con KMP: O(n)
- Complejidad total: O(n + m)
- Complejidad espacial: O(m), debido al arreglo LPS.

Esto resulta más eficiente que una búsqueda por fuerza bruta, que en el peor caso puede requerir O(n × m) comparaciones.

2. Algoritmo de Manacher
Para la segunda parte utilizamos el algoritmo de Manacher para encontrar el palíndromo más largo dentro de cada transmisión.

Primero se transforma la cadena agregando caracteres # entre sus caracteres originales. Esto permite manejar de la misma manera palíndromos de longitud par e impar.
Durante el recorrido se utiliza un arreglo P, donde cada posición almacena el radio del palíndromo encontrado alrededor de ese centro. También se mantienen las variables C y R, que representan el centro y el límite derecho del palíndromo que se ha extendido más hacia la derecha. Gracias a esta información, el algoritmo puede reutilizar resultados previamente calculados mediante posiciones espejo.

Si n es la longitud de la transmisión:
- Complejidad temporal: O(n)
- Complejidad espacial: O(n), debido a la cadena transformada y al arreglo P.

La ventaja de Manacher es que permite encontrar el palíndromo más largo en tiempo lineal, en lugar de revisar todas las posibles subcadenas.

3. Programación dinámica: Longest Common Substring

Para la tercera parte utilizamos programación dinámica para encontrar el substring común más largo entre transmission1 y transmission2.

Primero construimos una matriz DP donde DP[i][j] representa la longitud del substring común que termina en las posiciones correspondientes de ambas transmisiones.

Cuando los caracteres son iguales utilizamos la fórmula:

DP[i][j] = DP[i - 1][j - 1] + 1;

Esto permite extender una coincidencia que ya existía. Si los caracteres son diferentes:

DP[i][j] = 0;

Se coloca 0 porque se busca un substring, por lo que los caracteres deben aparecer consecutivamente.

Las variables mejorLongitud y mejorFinal nos permiten almacenar la longitud del mejor substring encontrado y la posición donde termina dentro de la primera transmisión.

Si n es la longitud de transmission1 y m la longitud de transmission2:

- Complejidad temporal: O(n × m)
- Complejidad espacial: O(n × m), debido a la matriz DP.